

Data Structure Lab8 : Queue 2022-2023

Topics

1. Create Queue Interface
2. Create Queue Using Array
3. Create Queue Using Linked Lists
4. Implement Basic Methods of Queue
 - isEmpty()
 - size()
 - first()
 - enqueue(E e)
 - dequeue()

Homework

1. Augment the ArrayQueue implementation with a new rotate() method having semantics identical to the combination, enqueue(dequeue()). But, your implementation should be more efficient than making two separate calls (for example, because there is no need to modify the size).

Data Structure Lab8 : Queue 2022-2023

```
        }<public class Lab8<T
        ;private T[] data
        ;private int size, front, rear

        }public Lab8(int capacity)
        ;data = (T[]) new Object[capacity]
        ;size = 0
        ;front = 0
        ;rear = capacity - 1
        {

        }()public void rotate
        } if (size > 0)
        ;()T temp = dequeue
        ;enqueue(temp)
        {
        {

        }public void enqueue(T element)
        ;if (size == data.length) throw new IllegalStateException("Queue is full")
        ;rear = (rear + 1) % data.length
        ;data[rear] = element
        ;++size
        {

        }()public T dequeue
        ;if (size == 0) throw new IllegalStateException("Queue is empty")
        ;T value = data[front]
        ;front = (front + 1) % data.length
        ;--size
        ;return value
        {

        }()public boolean isEmpty
        ;return size == 0
        {

        }()public int getSize
        ;return size
        {
```

2. Implement the clone() method for the ArrayQueue class.

Data Structure Lab8 : Queue 2022-2023

```
        }()public Lab8<T> clone
;Lab8<T> clonedQueue = new Lab8<>(data.length)
;Lab8<T> tempQueue = new Lab8<>(data.length)

        } while (!this.isEmpty())
        ;()T value = this.dequeue
        ;tempQueue.enqueue(value)
        ;clonedQueue.enqueue(value)
        {

        } while (!tempQueue.isEmpty())
        ;this.enqueue(tempQueue.dequeue())
        {

        ;return clonedQueue
        {
        }

        // السؤال ٣
    }<public class Lab8LinkedList<T
    ;private Node<T> front, rear
    ;private int size

    }<private static class Node<T
        ;T value
        ;Node<T> next

    } public Node(T value)
    ;this.value = value
    ;this.next = null
    {
    }

    }()public Lab8LinkedList
    ;front = rear = null
    ;size = 0
    {
```

3. Implement a method with signature concatenate(LinkedList Q2) for the LinkedList class that takes all elements of Q2 and appends them to the end of the original queue. The operation should run in $O(1)$ time and should result in Q2 being an empty queue.

Data Structure Lab8 : Queue 2022-2023

```
        } public Node(T value)
        {this.value = value
        ;this.next = null
        {
        {

        }()public Lab8LinkedList
        ;front = rear = null
        ;size = 0
        {

        } public void concatenate(Lab8LinkedList<T> Q2)
        ;if (Q2.front == null) return

        } if (this.rear != null)
        ;this.rear.next = Q2.front
        } else{
        ;this.front = Q2.front
        {

        ;this.rear = Q2.rear
        ;this.size += Q2.size
        ;Q2.front = Q2.rear = null
        ;Q2.size = 0
        {

        } public void enqueue(T value)
        ;Node<T> newNode = new Node<>(value)
        } if (rear == null)
        ;front = rear = newNode
        } else{
        ;rear.next = newNode
        ;rear = newNode
        {
        ;++size
        {

        }()public T dequeue
        ;if (front == null) throw new IllegalStateException("Queue is empty")
        ;T value = front.value
        ;front = front.next
        ;if (front == null) rear = null
        ;--size
        ;return value
        {

        }()public boolean isEmpty
        ;return size == 0
        ;
```

4. Use a queue to solve the Josephus Problem.

Data Structure Lab8 : Queue 2022-2023

```

Josephus Problem
    } public class Lab8Josephus
    {
        public static int josephus(int n, int k)
        {
            Lab8LinkedListQueue<Integer> queue = new Lab8LinkedListQueue<Integer>(n);
            for (int i = 1; i <= n; i++)
                queue.enqueue(i);

            while (queue.getSize() > 1)
            {
                for (int i = 1; i < k; i++)
                    queue.enqueue(queue.dequeue());

                queue.dequeue(); // Remove the k-th person
            }

            return queue.dequeue(); // The winner
        }
    }
}

```

5. Use a queue to simulate Round Robin Scheduling.

```

Round Robin Scheduling
    } public class Lab8RoundRobin
} public static void roundRobinScheduling(Lab8LinkedList<String> queue, int timeQuantum)
    } while (!queue.isEmpty())
    ;()String process = queue.dequeue
    ;System.out.println("Processing: " + process)
    Simulate processing with time quantum//
    } if (timeQuantum > 0)
;System.out.println(process + " completed its time slice.")
    {
        {
            {
                {

```